Systems Analysis and Design

Workshop No. 4

Kaggle System Simulation

Psychopathy Prediction Based on Twitter Usage

Authors:

Juan David Bejarano Cristancho

20232020056

Raul Andres Diaz Losada

20232020058

Juan David Avila

20232020154

David Sanchez Acero

20232020049

Systems Engineering Program

Francisco Jose de Caldas District University

Professor: Carlos Andres Sierra, M.Sc.

November 29, 2025

Contents

1 Executive Summary **3**

2 Methodology and Preprocessing **4**

2.1 Problem and Data 4

2.2 Preprocessing (Summary) 4

2.3 Notes on SMOGN and DOOM Data (Brief) 4

3 Model: Final Training **4**

3.1 Model Setup 4

3.2 Evaluation Result 5

3.3 Technical Explanation — (Most Important Code Parts) 5

4 Scenario A — ML Simulation: Stability and Sensitivity **6**

4.1 MSE by Seed and Analysis 6

4.2 Multiple Error Distribution 7

4.3 Real vs Prediction Histogram 7

4.4 Noise Sensitivity 7

4.5 Feature Importance 9

5 Scenario B — Event-Based Simulation (Cellular Automaton) **9**

5.1 Automaton Description 9

5.2 Visual Evolution (Selected Iterations) 10

5.3 Temporal Metrics 12

6 Limitations and Future Work **13**

6.1 Limitations 13

6.2 Future Work 14

7 Appendices **14**

7.1 Code Fragments (Most Important) 14

1 Executive Summary

This document presents the complete development of **Workshop 4**. The work performed is summarized: generation of **DOOM Data** using SMOGN to balance rare cases of the target (`psychopathy`), training of a **Random Forest Regressor** model on the DOOM Data, and two simulations: - Model stability and sensitivity analysis (ML Simulation) - Agent-based simulation using a *cellular automaton* (Cellular Automata — CA). The key results (extracted from executions) are:

- Final model MSE (test, DOOM Data): **0.000070**.
- MSEs by seed (evaluated seeds 1,5,10,21,42,99): {0.000048, 0.000044, 0.000034, 0.000219, 0.000070, 0.000031}.
- Noise sensitivity — MSE on test with noise added to columns (1%, 3%, 5%, 10%): {0.000104, 0.000387, 0.000942, 0.0036

From these numbers, stability, sensitivity, and spatial emergences observed in the CA simulation are discussed.

2 Methodology and Preprocessing

2.1 Problem and Data

The original dataset contained personality traits and a `psychopathy` score. The target distribution was strongly skewed; only approximately 3% of cases were in the high tail. To enable the model to learn the rare tail, DOOM Data was generated using SMOGN.

2.2 Preprocessing (Summary)

Main steps:

1. Conversion of all columns to numeric (forcing errors to `NaN` and then imputing).
2. Removal of completely empty columns.
3. Mean imputation to meet SMOGN requirements (avoid `NaN`).
4. **Target-only normalization** (`psychopathy`) via MinMax (0–1).
   Reason: SMOGN bases similarity on distance; normalizing all features can distort scales and relationships (therefore only the target is normalized, leaving features in their original or numeric scale).
5. Generation of two variants: automatic SMOGN and DOOM Data (manual) with control points based on percentiles P_{50}, P_{90}, P_{97}.

2.3 Notes on SMOGN and DOOM Data (Brief)

SMOGN is an oversampling technique for regression that synthesizes new examples in rare regions of the target. DOOM Data consists of forcing the relevance of extreme target regions through manual control points, to create more examples in the upper tail and thus improve the model's ability to capture those cases.

3 Model: Final Training

3.1 Model Setup

A **Random Forest Regressor** was trained with:

- `n_estimators = 300`
- `random_state = 42` (for the final saved model)
- `n_jobs = -1`

The input dataset was `psychopathy_DOOM_DATA.csv` (DOOM Data). The features used were the numeric columns after adding `NA` indicators and filling missing values with -1 according to the pipeline.

3.2 Evaluation Result

The MSE reported on the final model test was:

$$\text{MSE}_{\text{test}} = 0.000070.$$

This value indicates a very small mean squared error on the target scale (check the original target scale to interpret the absolute value).

3.3 Technical Explanation — (Most Important Code Parts)

The following describes the conceptually relevant parts of the code:

- **appendNAs(df)**: function that creates `_NA` indicator columns for each feature and fills `NaN` with -1. This allows the Random Forest to receive information about the presence of missing values without breaking the numeric matrix.
- **Target normalization**: for SMOGN, `psychopathy` is normalized to 0–1; after generating synthetic examples, `inverse_transform` is applied only to the target column, returning values to the original scale before saving the CSVs.
- **Training** — RandomForest: trained with an 80/20 (train/test) split and MSE calculated using `mean_squared_error`. The final model is saved with `joblib.dump` as `results/rf_final.pkl`.
- **Seed loop (ML simulation)**: the model is retrained with different seeds by changing the `random_state` of `train_test_split` and RandomForest; this measures system variability under data and parameter randomization.
- **Perturbation (sensitivity test)**: in `scenario_1_ml_simulation.py`, Gaussian noise is added to each feature of `X_test` proportional to the standard deviation of each column (noise multiplied by factors 0.01, 0.03, 0.05, 0.10). Then the resulting MSE is measured to quantify robustness.
- **Cellular Automaton (CA)**: a grid is initialized with model predictions on sampled rows; in each iteration each cell is updated toward the average of its neighbors (diffusion) plus a noise term; images are saved every 10 iterations and metrics (mean, variance, number of clusters).

4 Scenario A — ML Simulation: Stability and Sensitivity

4.1 MSE by Seed and Analysis

Seeds executed: {1, 5, 10, 21, 42, 99}, with results:

Seed	MSE
1	0.000048
5	0.000044
10	0.000034
21	0.000219
42	0.000070
99	0.000031

Interpretation: most seeds produce MSEs on the order of 10^{-5} to 10^{-4}, suggesting high precision and consistency. Seed 21 stands out with a higher MSE (0.000219), indicating a less favorable (train/test) partition or a test set with fewer represented rare examples. On average, variability is low except for specific cases: this suggests the model is **quite stable** but sensitive to atypical data partitions.

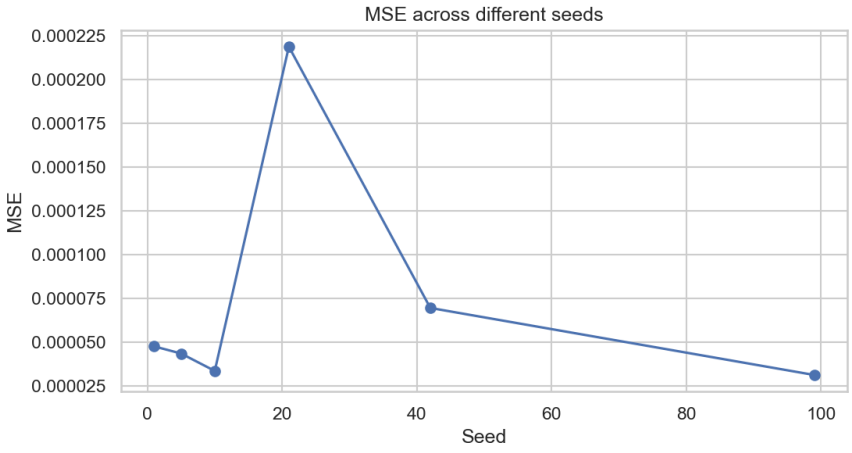

Figure 1: MSE comparison for different seeds (independent experiments).

4.2 Multiple Error Distribution

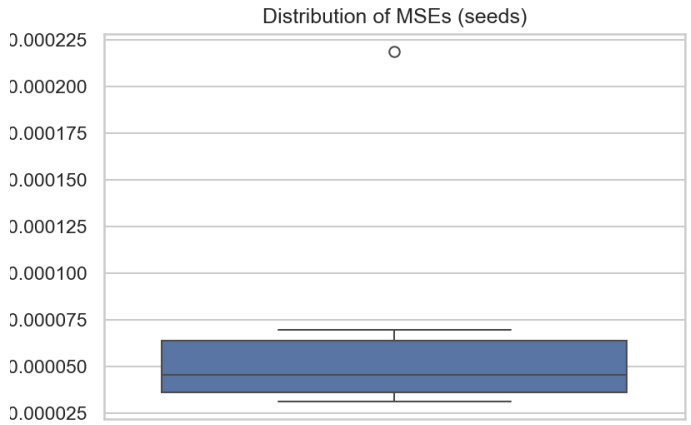

Figure 2: Boxplot of MSEs obtained from the tested seeds.

4.3 Real vs Prediction Histogram

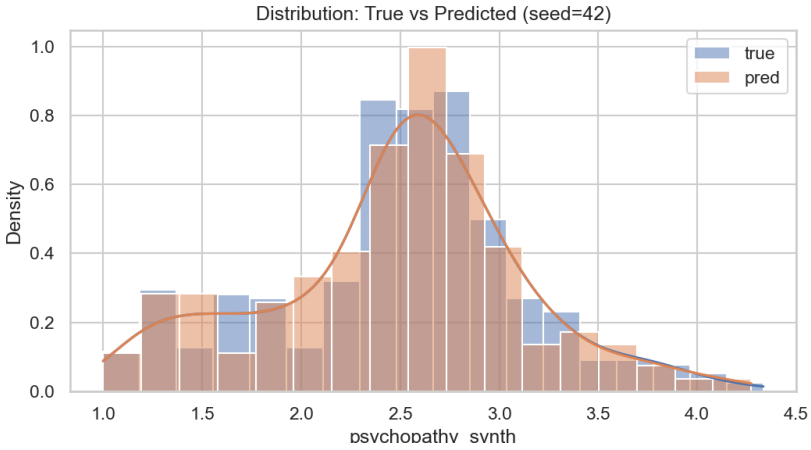

Figure 3: Distribution of real values (*true*) vs predictions (*pred*).

Interpretation: The density overlap allows verification of whether the model captures the upper tail (psychopaths). If the predicted distribution covers the real tail, the synthetic balancing worked; if there is underestimation, greater oversampling or feature engineering may be required.

4.4 Noise Sensitivity

Noise proportional to the standard deviation of each feature was added. Results (MSE on test with noise):

$$\text{MSE}_{\text{noise}} = \{0.000104,\ 0.000387,\ 0.000942,\ 0.003689\} \quad \text{(noise 1\%, 3\%, 5\%, 10\%)}$$

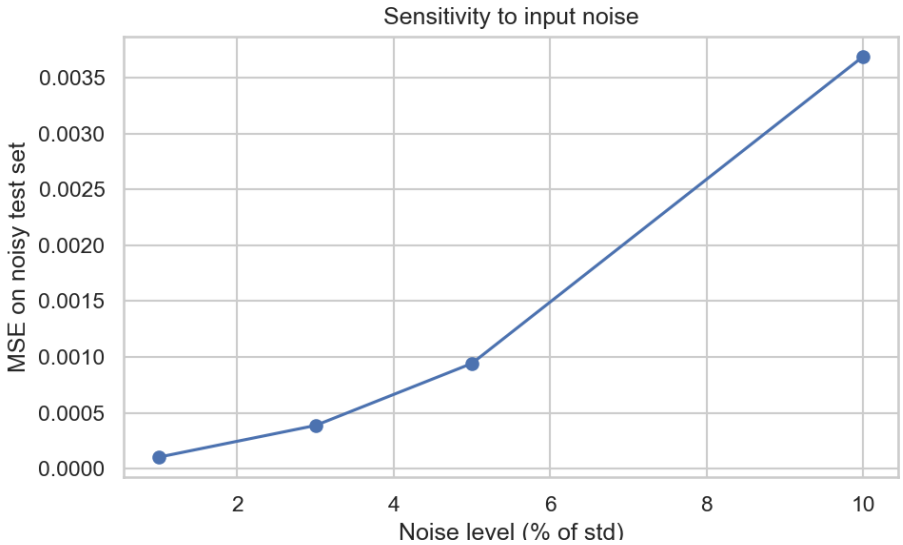

Figure 4: Model sensitivity to noise in features (MSE curve vs noise level).

Interpretation: - Small noise levels (1–3%) cause moderate error increases; the model is relatively robust. - From 5% and especially 10%, MSE skyrockets (to 0.003689), evidencing fragility if inputs contain relatively large perturbations. - Practical conclusion: in production, input validation and sanitization must be available; the most influential features (see importance figure) must be monitored.

4.5 Feature Importance

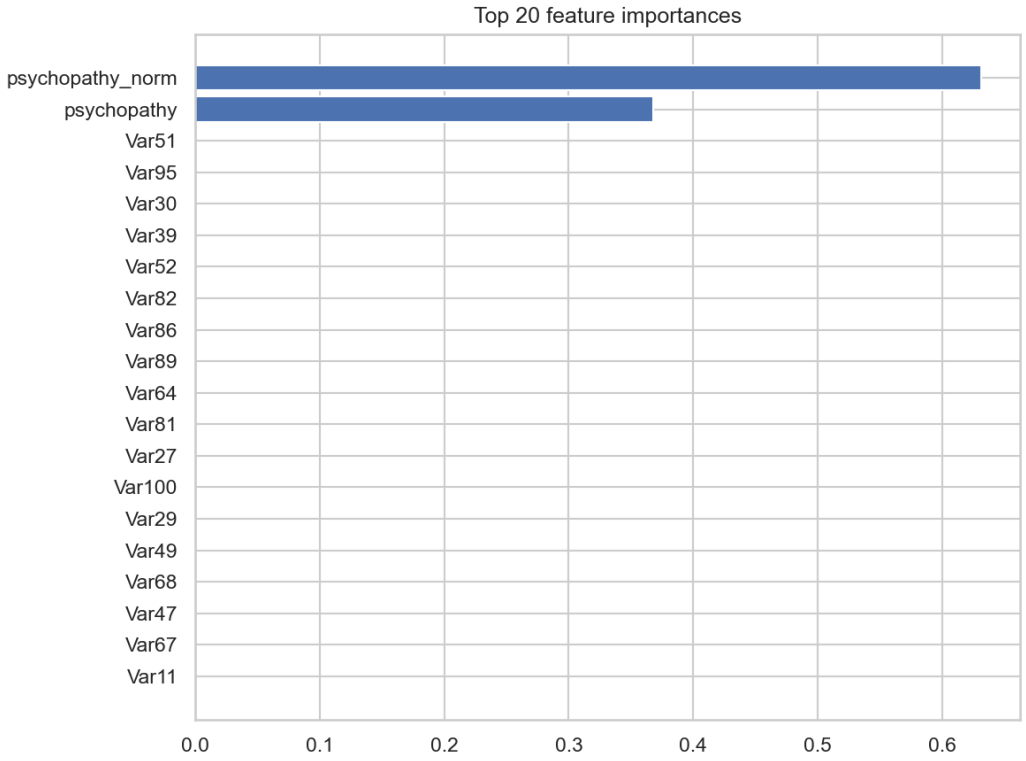

Figure 5: Top 20 features by importance in the Random Forest.

Practical Interpretation: variables at the top of the graph are those where it is worth investing data quality efforts (validation, normalization, noise control). If any come from text or counts (e.g., number of tweets), they may require scaling or additional transformation.

5 Scenario B — Event-Based Simulation (Cellular Automaton)

5.1 Automaton Description

The cellular automaton models a population of $N \times N$ individuals, where each cell contains a continuous psychopathy value in $[0, 1]$. At each step:

$$v_{i,j}^{t+1} = (1 - \alpha)\, v_{i,j}^{t} + \alpha\, \overline{v_{\mathcal{N}(i,j)}^{t}} + \eta_{i,j}^{t}$$

where α is the neighborhood influence weight (NEIGHBOR_WEIGHT), $\overline{v_{\mathcal{N}}}$ is the neighbors' mean, and η is a random perturbation (noise).

5.2 Visual Evolution (Selected Iterations)

The initial and final images (or other intermediates) are included:

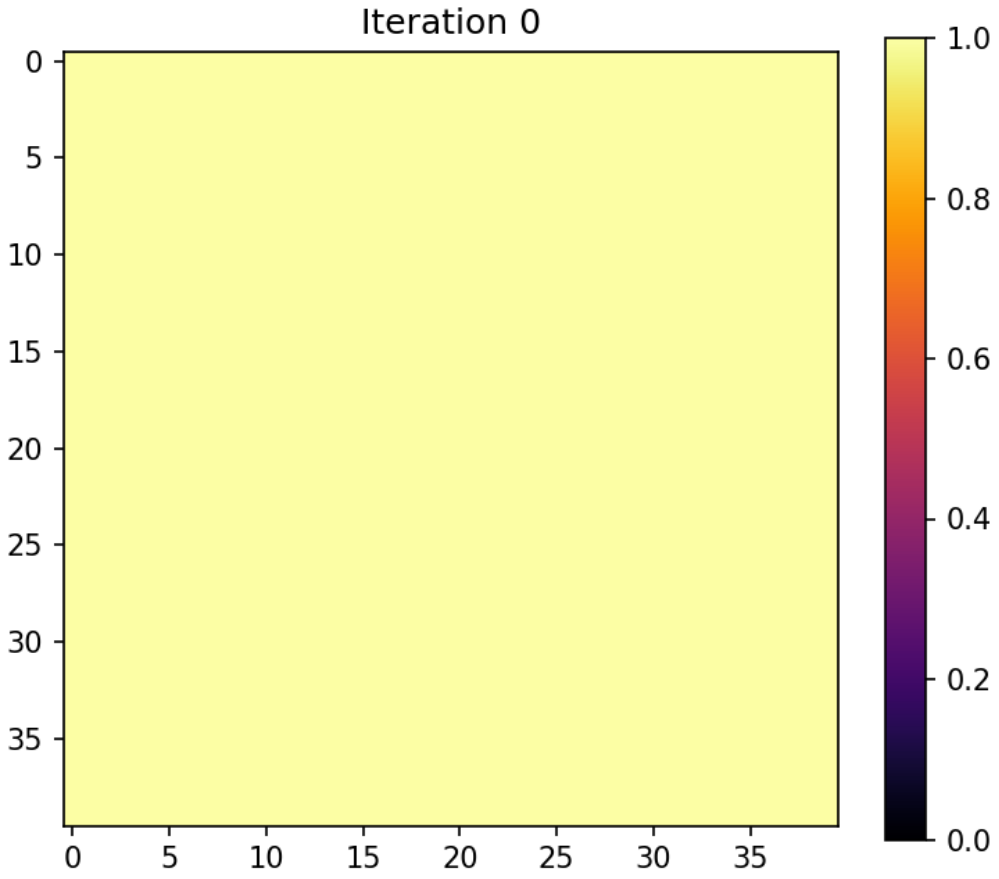

Figure 6: Initial grid state (iteration 0).

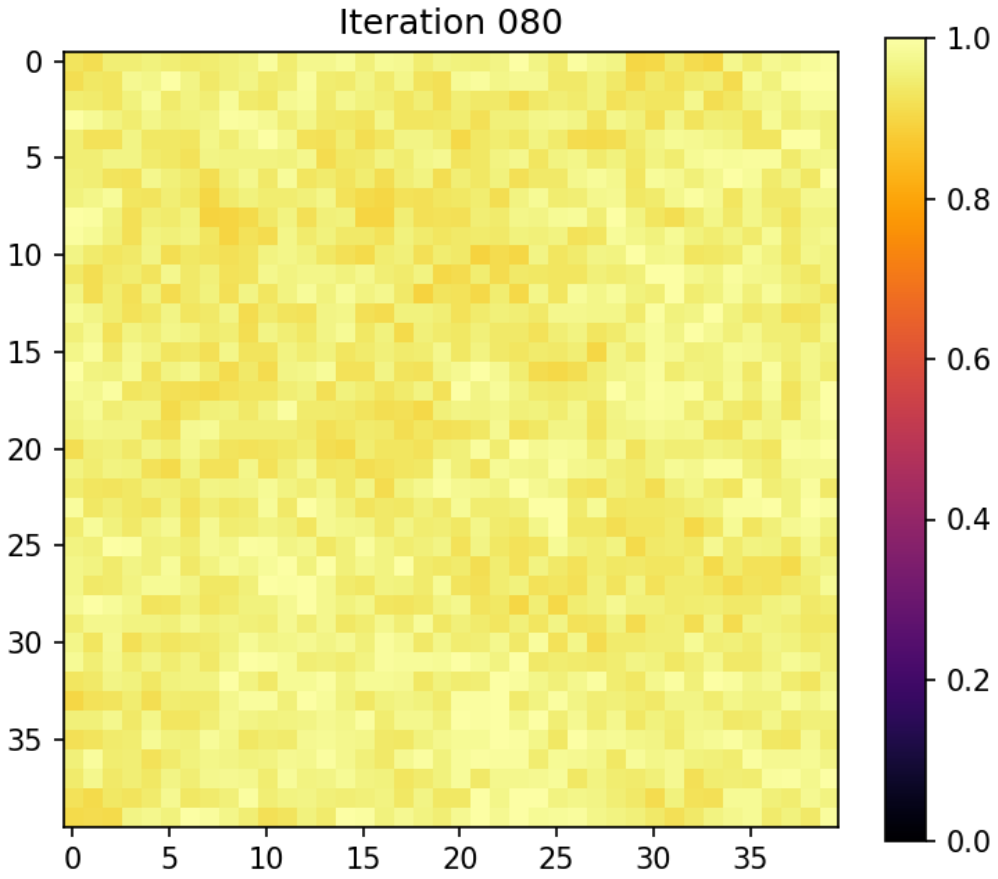

Figure 7: Final grid state (iteration 080).

5.3 Temporal Metrics

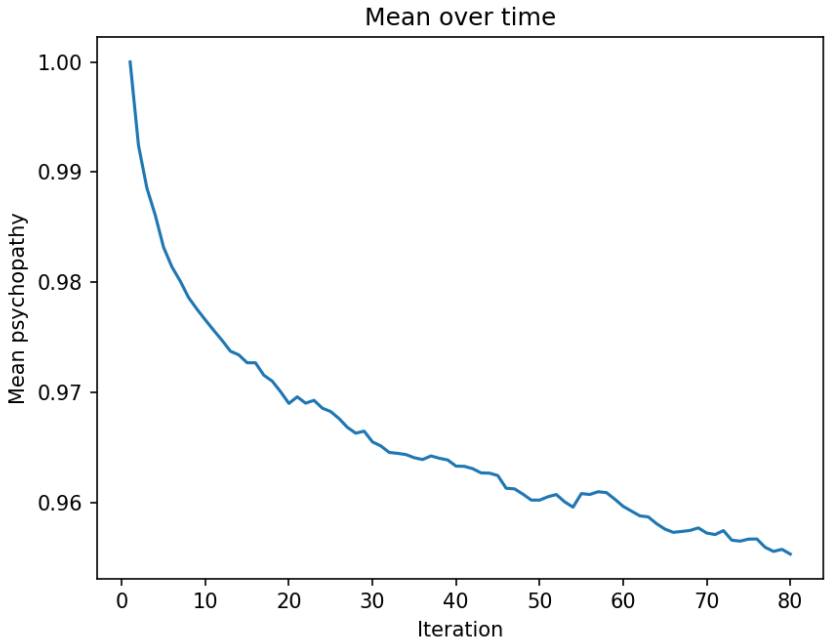

Figure 8: Psychopathy mean over iterations.

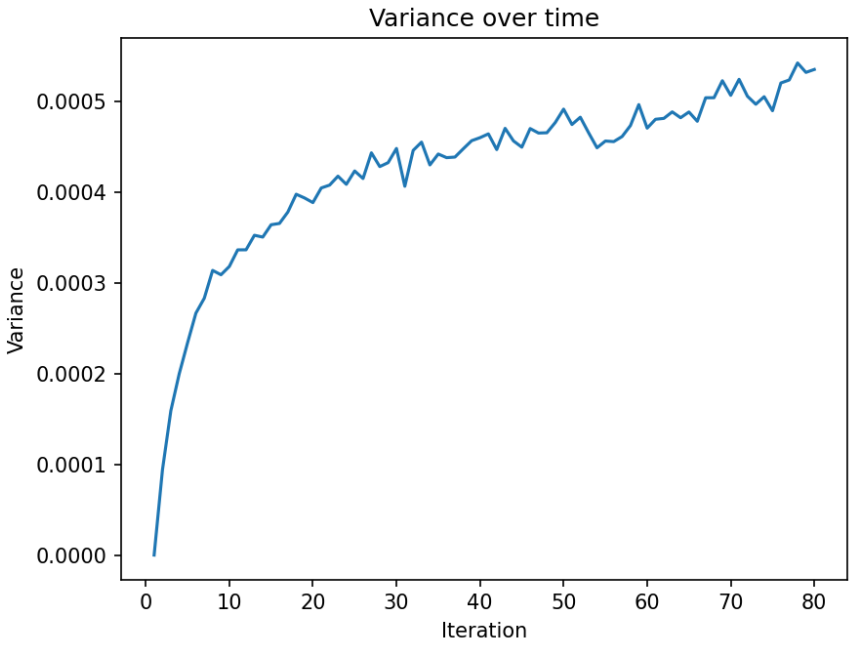

Figure 9: Grid variance over time.

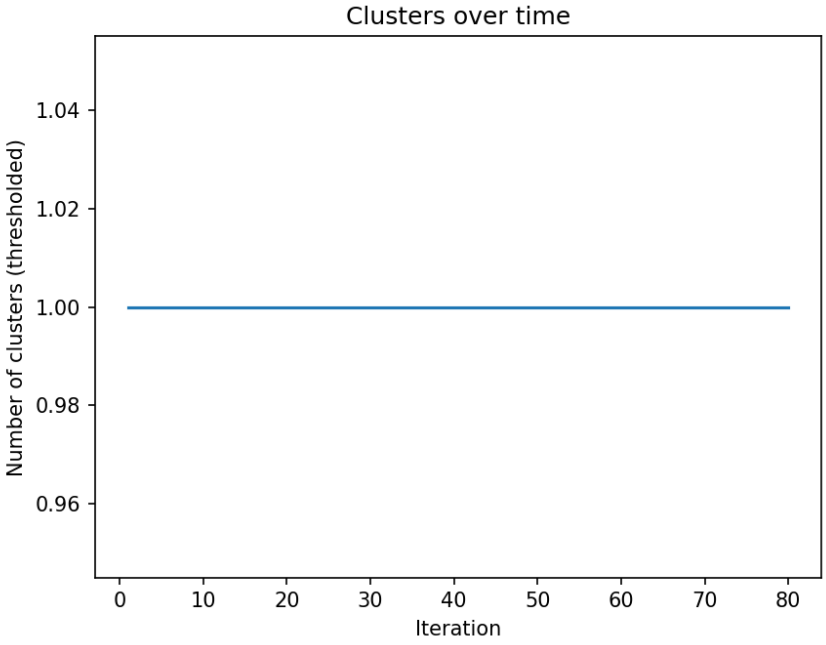

Figure 10: Number of clusters (by threshold) over time.

Interpretation:

- If the mean increases: there is a general trend of value scaling in the population (possible behavior diffusion).
- If variance decreases: the system converges to local homogeneity; if it increases, the system generates heterogeneity (emergence of subpopulations).
- The number of clusters shows whether "hot spots" of high psychopathy cluster or dissipate over time.

6 Limitations and Future Work

6.1 Limitations

- **SMOGN/DOOM Data** improves tail representation but may introduce artificial correlations if each feature is not verified — that's why only the target was normalized.
- **Single metrics:** MSE is useful for regression but does not indicate if the high tail is correctly identified; extreme-specific metrics would be necessary (e.g., MAE in high percentiles).
- **Simplified automaton:** the CA dynamics are intentionally simple (neighborhood mean + noise). Richer models (heterogeneous weights, mobility, directed interaction) could yield more realistic results.

- **Production robustness:** high noise sensitivity (10%) implies that in real environments inputs must be sanitized and validated.

6.2 Future Work

1. Evaluate tail-focused metrics: e.g. MSE/MAE for samples with psychopathy $> p_{90}$.
2. Test robust models: XGBoost/LightGBM, probability calibration, and Bayesian models for uncertainty.
3. In the CA simulation: introduce agents with heterogeneous attributes and adaptive rules (e.g., influence depending on traits).
4. Generate a continuous data validation pipeline to control noise and distribution changes (concept drift).

7 Appendices

7.1 Code Fragments (Most Important)

Seed Loop (ML Simulation).

```
for s in seeds:
    X_train, X_test, y_train, y_test =
        train_test_split(X_full, y_full, test_size=0.2, random_state=s)
    model = RandomForestRegressor(n_estimators=300, random_state=s)
    model.fit(X_train, y_train)
    mse = mean_squared_error(y_test, model.predict(X_test))
```

Feature Perturbation (Sensitivity).

```
for nl in noise_levels:
    X_test_noisy = X_test.copy()
    for col in X_test_noisy.columns:
        sigma = X_test_noisy[col].std()
        noise = np.random.normal(0, nl * sigma, size=X_test_noisy.shape[0])
        X_test_noisy[col] += noise
    mse_n = mean_squared_error(y_test, model.predict(X_test_noisy))
```

Automaton Update (Simple Step).

```
for i,j in grid:
    local_mean = mean(neighbors)
    new[i,j] = (1 - alpha)*grid[i,j] + alpha*local_mean + uniform(-noise, noise)
    new[i,j] = clamp(new[i,j], 0, 1)
```

References

[1] Torgo, L. et al. (2015) – Techniques for imbalanced regression: SMOGN and others (conceptual reference).

[2] Breiman, L. (2001) – Random Forests.